# GradeOps

AI-Powered Exam Grading & Results Management Platform

Project Technical Report

May 2026 | Version 1.0

Krish Patel | Ayush Bansal | Lalit Deshmane

Python 3.12 | FastAPI | React 19.2 | Qwen2-VL OCR | PyMuPDF | JWT Auth

# Abstract

GradeOps is a full-stack, AI-assisted examination grading platform that automates the evaluation of handwritten and printed answer sheets. The system combines a Vision-Language Model (Qwen2-VL-2B-OCR) for optical character recognition with a structured marking scheme evaluator to produce per-question scores, matched and missing criteria, and natural-language reasoning for each submission. The platform is accessible through a single-page React dashboard and backed by a FastAPI REST API with JWT authentication. All data is persisted as flat JSON files, eliminating the need for a dedicated database.


# 1. Introduction

## 1.1 Background

Manual evaluation of written examinations remains a significant bottleneck in academic institutions. A single instructor may be responsible for grading dozens to hundreds of answer sheets per examination cycle — a process that is time-consuming, subjective, and prone to inconsistency across evaluators. The rise of large Vision-Language Models capable of reading handwritten text creates an opportunity to partially automate this workflow while preserving the nuance of rubric-based grading.

## 1.2 Problem Statement

Traditional grading workflows suffer from three core problems:

- **Speed:** Manual evaluation of a 30-question paper for 60 students can exceed 40 person-hours.
- **Consistency:** Subjective judgement varies between evaluators and even within a single evaluator across a session.
- **Traceability:** Paper-based records lack structured storage, making historical analysis and appeals difficult.

## 1.3 Objectives

- Automate OCR extraction of handwritten and printed answer sheets using a Vision-Language Model.
- Evaluate extracted text against instructor-defined marking schemes with configurable strictness.
- Provide per-question scoring with matched/missing criteria and reasoning.
- Deliver a secure, instructor-facing REST API and a responsive single-page dashboard.
- Accept answer sheets and marking schemes in both image and PDF formats.

# 2. System Architecture

GradeOps is structured as a decoupled two-tier system: a React single-page application communicating with a FastAPI backend via authenticated REST calls. The backend itself is further decomposed into an API orchestration layer and an AI inference pipeline.

## 2.1 Component Overview

| | |
|---|---|
| **Instructor Portal** | React 19.2 SPA — single-page dashboard with a persistent sticky sidebar and collapsible accordion sections for Exams, Students, and Results. |
| **API Orchestrator** | FastAPI backend exposing six route modules (Auth, Exams, Students, OCR, Evaluate, Submissions) behind JWT Bearer authentication. |
| **File Normalisation** | PyMuPDF (fitz) renders multi-page PDFs to stacked PIL images at 200 DPI; JPEG/PNG inputs are opened directly via Pillow. |
| **Vision OCR Engine** | Qwen2-VL-2B-OCR (HuggingFace Transformers + PyTorch) extracts plain-text transcripts from normalised images, up to 2 048 tokens per sheet. |
| **Evaluation Engine** | Compares extracted transcript against each question's expected_points and keywords arrays, applying a strictness modifier to compute partial or full marks. |
| **JSON File Store** | All records (users, exams, students, schemes, submissions, evaluations) are persisted as structured flat JSON files under data/ — no external DB required. |

## 2.2 Request Lifecycle — Answer Sheet Grading

| Step | Actor | Action |
|---|---|---|
| 1 | Frontend | Instructor submits exam_id, student_id, and answer sheet file via multipart POST /evaluate/grade |
| 2 | Auth Guard | JWT token extracted from Bearer header; instructor identity resolved |
| 3 | Route Handler | File saved to uploads/; marking scheme retrieved from JSON store |
| 4 | Image Loader | If PDF: PyMuPDF renders pages → stacked PIL image. If image: PIL opens directly |
| 5 | OCR Service | Qwen2-VL-2B-OCR tokenises image and generates text transcript |
| 6 | Evaluator | Transcript scored per-question against expected_points / keywords with strictness weighting |
| 7 | Store | Submission and evaluation records written to data/submissions/ and data/evaluations/ |
| 8 | Response | JSON payload returned: scores, percentage, question_wise breakdown, overall_reasoning |

# 3. Technology Stack

## 3.1 Backend

| | |
|---|---|
| **Python 3.12** | Primary runtime for all backend services and data processing. |
| **FastAPI** | Async REST framework with automatic OpenAPI/Swagger documentation at /docs. |
| **Uvicorn** | ASGI server for serving the FastAPI application. |
| **Pydantic v2** | Request/response schema validation and environment configuration via pydantic-settings. |
| **python-jose** | JWT token encoding, decoding, and expiry verification. |
| **Passlib + bcrypt** | Password hashing and verification for instructor accounts. |
| **PyMuPDF (fitz)** | PDF rendering and text extraction — converts PDF pages to PIL images for OCR. |
| **Pillow** | Image loading, format conversion, and multi-page stitching. |

## 3.2 AI / ML

| | |
|---|---|
| **Qwen2-VL-2B-OCR** | Vision-Language Model by JackChew (HuggingFace) — fine-tuned for document OCR on the Qwen2-VL architecture. |
| **Transformers** | HuggingFace library for model loading, tokenisation, and autoregressive inference. |
| **PyTorch** | Deep learning runtime; CUDA GPU acceleration supported (CPU fallback available). |
| **Accelerate** | HuggingFace utility for automatic device placement (CPU / single GPU / multi-GPU). |

## 3.3 Frontend

| | |
|---|---|
| **React 19.2** | Component-based SPA framework with hooks for state and side-effects. |
| **React Router v7** | Client-side routing with ProtectedRoute guards; legacy /exams /students /results redirect to /dashboard. |
| **Vite 8.0** | Next-generation frontend build tool with HMR (Hot Module Replacement) for fast development. |
| **Axios** | Promise-based HTTP client; all API calls attach a Bearer token from AuthContext. |

| | |
|---|---|
| **IntersectionObserver** | Used in Sidebar.jsx to detect which section is in the viewport for scroll-spy active highlighting. |

# 4. Key Features & Implementation

## 4.1 Marking Scheme Upload

Instructors upload a marking scheme linked to a specific exam. The scheme follows a structured JSON format defining, for each question: **question_no**, **max_marks**, **expected_points** (list of expected answer criteria), **keywords** (key terms that signal understanding), and a **strictness** level. Both raw JSON files and text-extractable PDFs are accepted. For PDFs, PyMuPDF's *page.get_text()* extracts embedded text before JSON parsing; image-based (scanned) PDFs are rejected with a descriptive error.

Scheme JSON structure:

```
{ "exam_name": "...", "total_marks": 30, "questions": [ { "question_no": 1,
"max_marks": 10, "expected_points": ["..."], "keywords": ["..."], "strictness":
"medium" } ] }
```

## 4.2 AI Answer Sheet Grading

Answer sheets are submitted as JPEG, PNG, or PDF uploads. The grading pipeline operates in four stages: (1) file normalisation via PyMuPDF/Pillow, (2) OCR transcript generation by Qwen2-VL-2B-OCR, (3) per-question evaluation against the marking scheme, and (4) aggregation into a final score, percentage, and structured question-wise breakdown. Each question result includes **awarded_marks**, **matched_points**, **missing_points**, and a **reasoning** string explaining the score.

## 4.3 Single-Page Instructor Dashboard

The frontend consolidates the Exams, Students, and Results workflows into a single scrollable page with collapsible accordion sections. A persistent sticky sidebar uses the browser's *IntersectionObserver* API to automatically highlight the active section as the instructor scrolls. Clicking a sidebar item smoothly scrolls to the corresponding section anchor. The student list includes a live search bar filtering by name or student ID without a server round-trip.

## 4.4 Authentication & Security

- **Password hashing:** bcrypt via Passlib — plaintext passwords are never stored.
- **JWT tokens:** HS256-signed tokens with configurable expiry (default 60 min) issued on successful login.
- **Auth guard:** A FastAPI Depends() decorator on every protected route extracts and verifies the Bearer token, resolving the instructor identity.
- **CORS:** Configured in main.py — origins should be restricted to the frontend URL in production.

# 5. API Reference

| Method | Endpoint | Description |
|---|---|---|
| POST | /auth/register | Register a new instructor account |
| POST | /auth/login | Authenticate and receive a JWT token |
| POST | /exams/create | Create a new exam record |
| GET | /exams/ | List all exams for the authenticated instructor |
| POST | /students/create | Register a new student |
| GET | /students/ | List all students |
| POST | /evaluate/upload-scheme | Upload a marking scheme (JSON or text PDF) |
| POST | /evaluate/grade | Grade an answer sheet (image or PDF) via OCR + evaluator |
| GET | /submissions/exam/{exam_id} | Retrieve all evaluation results for a given exam |
| GET | /submissions/student/{student_id} | Retrieve all evaluation results for a given student |

*All endpoints except /auth/* require Authorization: Bearer <token> in the request header.*

## 6. Environment Configuration

Runtime behaviour is controlled via a .env file in the backend/ directory:

| | |
|---|---|
| **MODEL_NAME** | HuggingFace model ID for OCR inference (default: JackChew/Qwen2-VL-2B-OCR) |
| **MAX_NEW_TOKENS** | Maximum token output per OCR pass (default: 2048) |
| **UPLOAD_DIR** | Directory where answer sheet files are saved (default: uploads/) |
| **DATA_DIR** | Root directory for all JSON data stores (default: data/) |
| **JWT_SECRET_KEY** | Secret key for HS256 token signing — must be changed in production |
| **ACCESS_TOKEN_EXPIRE_MINUTES** | JWT token lifetime in minutes (default: 60) |

## 7. Conclusion & Future Work

GradeOps demonstrates that a Vision-Language Model can meaningfully automate structured exam grading when paired with a well-defined marking scheme. The current implementation covers the complete instructor workflow — from exam creation and student registration to AI-graded submissions and results lookup — within a lightweight, dependency-minimal stack.

Identified areas for future development include:

• **Database backend:** Migrate from flat JSON to PostgreSQL or SQLite for concurrent access and query performance.

• **Bulk upload:** Accept a ZIP of answer sheets and process them asynchronously with a task queue (Celery / ARQ).

• **Larger OCR models:** Evaluate Qwen2-VL-7B or GPT-4o Vision for improved handwriting recognition accuracy.

• **Scanned PDF scheme support:** Add OCR-to-JSON extraction for image-based marking scheme PDFs.

• **Appeals workflow:** Allow instructors to manually override individual question scores with a reason.

• **Analytics:** Add class-level statistics — score distributions, question difficulty analysis, cohort comparisons.